

Build-fix provenance: capturing *what makes each family build*, machine-readably

Model: Claude Opus 4.8 (1M context) · drafted 2026-06-05 · **Status:** direction set — consumer ("declarative manifest + tiny runner") and scenario-B capture ("auto-detect → confirm") decided 2026-06-05; design still open on the runner's privilege contract, layer coherence, and storage.

1. The problem in one paragraph

`gflib-build` is an interactive dashboard that orchestrates building the whole Google Fonts library. When a family fails to build, we *do something* — improve its dependency cohort, install a missing system package, add a pre-build step, fix a mis-cased filename — and the family starts building correctly. **But the knowledge of what was done to make it build is not consolidated anywhere.** It is scattered across several stores, in different formats, some of it transient or run-scoped, and at least one whole category (system packages) is **not recorded at all**. There is no single, machine-readable, per-family "build recipe" — a Makefile/lockfile-equivalent — that a much simpler, non-interactive build system could consume to reconstruct the exact environment and reproduce the build.

2. What "a fix" actually is — the scenarios

When a failing family becomes a building family, the change falls into one of these kinds (extends the A/B/C Felipe sketched, grounded in the real failure taxonomy in

`rust/src/classify.rs`):

- **A — Dependency-cohort fix.** The family was resolved against an inadequate set of Python build dependencies. We improved the cohort / the effective requirements (include-expansion of `-r` files, filtering out QA tools, family pins overriding the base toolchain, `PIN_OVERRIDES`, or auto-relaxing an unsatisfiable pin). Maps to buckets `broken dependency venv`, `dependency conflict`, `dependency install failed`, `pip resolution too deep`, `build needs setuptools`, `misconfigured requirements`.
- **B — Missing system software.** The build needed a native library or system package (e.g. `apt install libcairo2-dev pkg-config`) **outside** the Python venv. The tool *detects* this (bucket `missing system library`, `is_auto_retry = false`) and tells the user, who installs it on the host.
- **C — Source / pre-build fix.** The source tree had to change before the builder could run: generate sources, decompress/merge UFOs, stage a correctly-cased filename, inject OpenType features. Registered as ordered shell commands in `build_rules.json`.
- **D — Provenance / config fix.** Wrong commit pinned, or a `config.yaml` missing/incorrect → corrected `METADATA.pb` / added an override `config.yaml` in `google/fonts`.
- **E — Toolchain choice.** Which compiler (`fontc` / `fontmake`) and orchestrator (`builder2` / `builder3`), at which exact version, produced the artifact.

3. Where fix-knowledge lives today (the fragmentation)

Fix kind	Where it's stored now	Format	Durability
A Python deps / cohort	<code>state.json</code> → per-family <code>cohort</code> key + top-level <code>cohort_reqs</code> / <code>cohort_members</code>	JSON, hash-keyed	Run-scoped. The <i>effective</i> assembled requirements (<code>venv::assemble_requested</code>) is computed transiently; only <code>--effreq</code> prints it. Relaxations are in memory (<code>VenvManager.relaxations</code> , "for the UI"); venv readiness is a <code>.gflib-installed</code> hash , not a readable spec.
B System packages	<i>Nowhere durable.</i> Only a generic, guessed hint in <code>classify.rs</code>	prose hint	None. Not the actual package, and lost the moment the build succeeds.
C Pre-build steps	<code>build_rules.json</code> (<code>rules.<slug>.pre_build</code>)	JSON, ordered shell cmds	Version-controlled ✓ — the one fix kind we capture well.
D Source + config	<code>google/fonts</code> <code>METADATA.pb</code> + override <code>config.yaml</code>	protobuf / YAML	Version-controlled, different repo , not joined to the build record.
E Compiler / builder	<code>state.json</code> per-family <code>compiler_version</code> , <code>builder</code> , <code>builder_version</code> (M0)	JSON	Run-scoped.

The shape of the gap: (1) no single artifact answers "*what does a minimal builder need to reproduce family X?*"; (2) A/E are run-scoped and the effective requirements (A) are never written out as a spec; (3) **system packages (B) — the most "environmental" dependency — are not recorded at all**; (4) even where data exists it is keyed by internal hashes and split across repos, so a "dumb" downstream builder cannot read it.

4. Direction (decided forks)

4.1 Consumer = a declarative manifest + one tiny generic runner

The "much simpler build system" is concrete: a **pure-data manifest** (no logic) plus a single **standalone runner** (`run_manifest`) that interprets it. The runner **must not** depend on `gflib-build`, the cohort hashing, or the live dashboard — given only a family's manifest and a checkout of the upstream source, it reconstructs the environment and builds. Because the manifest is data, the *same* artifact also feeds dashboards and the M5 Python-free burn-down (it's queryable, not just executable).

System packages: verify-and-assert, never install (decided 2026-06-05). The runner stays **unprivileged and distro-agnostic** — it *checks* the required packages are present (`pkg-config --exists` , `ldconfig -p` , ...) and **fails fast printing the exact** `apt` / `dnf` **line** if not. Installation is a human/CI step, never something the runner does with `sudo`.

4.2 Two-layer structure (kills the dedup problem)

Most of the environment is **cohort-level and shared**; only a few things are family-level. Split accordingly:

- **Cohort environment manifest** — `env/<cohort>.yaml` : `system_packages` , the **fully resolved** Python lockfile (`pip freeze` of the cohort venv after a green build — the whole transitive closure, exact versions), the relaxations/pin-overrides applied, the base interpreter version. Shared by every family in the cohort.
- **Family recipe** — `families/<slug>.yaml` : source repo + commit + config, pre-build steps, toolchain (compiler + orchestrator + versions), a pointer to its cohort, and the build verdict. References — does not copy — the cohort env.

This mirrors how cohorts already factor shared requirements, makes "what's Python-bound" measurable per cohort, and means a 200-family cohort has **one** env manifest, not 200 copies.

4.3 Generated vs curated layers, and how they merge

- **Auto-derived** (the tool emits each run): resolved pins, compiler/orchestrator versions, source/commit/config, pre-build (read from `build_rules.json`).
- **Curated** (version-controlled, human-owned, sibling to `build_rules.json`): the irreducible decisions — `system_packages` (B) and any pre-build steps (C).
- The **emitted manifest = merge(auto-derived, curated)**. Same split, and same sync discipline, the dashboard already uses.

4.4 Scenario-B capture = auto-detect → confirm → curated

When a `missing system library` failure **clears on a later retry**, a detector proposes the package — by parsing the native failure (`pkg-config` "Package X was not found", a missing `.so` , or `ldd` over the built native extensions) and mapping it to a package name. The user **confirms or corrects** it; the result is written to the curated `system_packages` file (cohort-keyed) and folds into that cohort's env manifest. Machine proposes, human ratifies — best accuracy, and it captures at the exact moment the knowledge exists.

Illustrative sketch (shape only — not a committed schema)

```
# env/c-1a2770e61902.yaml — cohort environment (shared by all its families)
cohort: c-1a2770e61902
base_python: "3.13.1"
system_packages:          # scenario B — curated (auto-detect → confirm)
  - libcairo2-dev
  - pkg-config
resolved_requirements:    # scenario A — `pip freeze`, full transitive closure
  - fontmake==3.12.1
  - gftools==0.9.996
  - compprefor==0.6.0      # PIN_OVERRIDE (compprefor>=0.5.6)
  - fonttools==4.55.0
# ... the complete lockfile ...
relaxations:              # currently in-memory only
  - "somepkg: pinned version unavailable on PyPI; pin dropped"
filtered_out: [fontbakery, "gftools[qa]"]
```

```
# families/ofl__cairoplay.yaml — family recipe
family: ofl/cairoplay
cohort: c-1a2770e61902      # → env/ above
source:
  repo: https://github.com/.../Cairo
  commit: abc1234
  config: sources/cairoplay.yaml
pre_build:                 # scenario C — from build_rules.json
  - python3 scripts/makeplay.py sources/Cairo.glyphs sources/CairoPlay.glyphs
toolchain:                 # scenario E — from M0
  compiler: "fontc 0.x.y (git deadbee)"
  orchestrator: "gftools-builder2 0.9.996"
build:
  status: built
  reproduces_shipped: true  # provenance verdict (see §8)
migration:                 # north-star ledger
  python_blockers: [venv, pre_build, builder2]
  readiness_level: L4
```

5. Open design questions (the now-sharper ones)

1. ~~Runner privilege & system packages.~~ **DECIDED 2026-06-05: verify-and-assert, never install** — see §4.1. The runner is unprivileged and distro-agnostic; it checks presence and prints the exact install command on failure.
2. **Layer coherence / drift.** When a cohort's resolved pins change (toolchain bump) but the curated `system_packages` don't, how do we detect drift and keep the merged manifest coherent? Likely the same validate-before-sync discipline as `sync_dashboard.py`.

3. **Storage & promotion.** Curated layer lives version-controlled here (next to `build_rules.json`). The merged/emitted manifest — kept only in the build dir, or **promoted into version control as the durable record**? The "build registry as permanent record" policy argues for promoting it.
4. **Format.** YAML (hand-editable, matches `config.yaml`) for the curated/human files; JSON acceptable for emitted artifacts (matches `state.json`).
5. **`build_rules.json` relationship.** Subsume it into the family recipe (pre-build becomes a section) or keep it as the source the recipe references? *Recommend: reference now, subsume later.*
6. **Verification granularity.** "Builds + produces the expected files" vs **byte-identical** to the recorded output (ties to M3 equivalence). Start with the former; aim for the latter.

6. Staging (MVP-first)

- **Stage 0 — consolidation, no new capture.** Emit cohort env + family manifests purely from data we *already have*: `pip freeze` the cohort venv after a green build, versions from M0, source/commit/config from `METADATA.pb`, pre-build from `build_rules.json`. Ship the generic runner. (`system_packages` present, possibly empty.) Immediate value; proves the contract.
- **Stage 1 — scenario B.** Add auto-detect → confirm on `missing system library` resolution; populate cohort `system_packages`; merge.
- **Stage 2 — round-trip verification.** Run the runner in a clean environment (fresh venv; optionally a throwaway container for true host cleanliness) and confirm reproduction. §7.
- **Stage 3 — migration ledger.** Surface per-cohort Python-blocker status from the manifests; wire to the M5 burn-down and the dashboard.

7. Verification (acceptance)

A manifest is **trusted only when the runner reproduces the build from it in a clean environment** — manifest + round-trip check land together, never the manifest alone.

Stage-2 acceptance: the runner rebuilds the family from its manifest; stretch goal: byte-identical to the recorded build output (feeds M3).

8. Relationship to existing work

- `build_rules.json` — already the machine-readable home for fix kind **C**; the manifest generalises it to A/B/D/E. The runner is the concrete "simpler build system."
- **M0 provenance** (`provenance.rs`) — already records **E**; the manifest *consumes* it.
- **"Build registry as permanent record" policy** — the manifest is the concrete realisation of *record all build parameters so the library can be rebuilt from scratch*. Stage 0 leans entirely on existing data (`pip freeze` + M0 + `build_rules` + `METADATA`).
- **North-star milestones (M0-M7)** — the `migration` section is the per-family/per-cohort evidence for the L0-L6 ladder and the M5 burn-down; every Python field is a visible blocker.
- **Full-Library Source-Provenance Review**
(`GoogleFonts/data/provenance_review/PLAN.md`) — **complementary, different axis**. That

answers "does the pinned commit produce the shipped binary?" (source → artifact); this answers "what environment reproduces the build?" (environment → artifact). They meet at `source` and `reproduces_shipped`; cross-reference, don't merge.

9. Prior art: Debian (provenance & reproducible builds)

Debian's source-provenance and downstream-patch discipline is the closest existing model for this work. **Adopt the schemas; do not bulk-package the library** (see the verdict below).

Debian artifact	Purpose	Our equivalent
<code>debian/copyright</code> (DEP-5)	machine-readable per-file copyright/license + <code>Source:</code> URL	source-provenance review (<code>provenance_review/PLAN.md</code>)
<code>debian/patches/</code> + DEP-3 headers	downstream patches vs <i>pristine</i> upstream, each tagged <code>Origin</code> / <code>Forwarded</code> / <code>Author</code> / <code>Description</code>	our fixes: <code>build_rules.json</code> , override <code>config.yaml</code> , pin-overrides
<code>.buildinfo</code> + Reproducible Builds	records the exact build environment for bit-for-bit reproduction	the cohort env manifest / <code>pip freeze</code> lockfile (§4.2)
<code>Build-Depends</code> (<code>debian/control</code>)	declarative system/library build deps	scenario-B <code>system_packages</code>
<code>debian/watch</code> + <code>uscan</code>	upstream location + new-version detection	"scan recent upstream commits for new families"

Adoption (low cost, high alignment): (1) replace `build_rules.json`'s freeform `note` with **DEP-3-style structured headers** (`Origin`, `Forwarded`, `Reason`); (2) model the cohort env manifest on `.buildinfo`; (3) treat `system_packages` as `Build-Depends`.

Overlap. Debian already ships some GF-distributed families (`fonts-roboto`, `fonts-noto-*` — genuinely Google-originated — plus many GF *redistributes* but didn't originate, e.g. `fonts-lato`, `fonts-cabin`). Most GF families originate from independent foundries (GF is a distribution channel), so for the majority **Debian's upstream and ours are the same third party** — which is exactly why sharing *provenance* + *build recipes* (not packages) is the high-leverage move.

Verdict — help or hinder? Adopting the *methods* **strongly helps**: DEP-5 / DEP-3 / `.buildinfo` / `Build-Depends` are source-provenance + reproducible-build discipline, which is our actual mission. **Bulk-packaging the ~2,000 families as `.deb` s hinders**: a perpetual maintenance treadmill that does nothing for the fontc/Rust north star (a `.deb` ships a built binary; it doesn't reach L5/M5), and Debian's NEW-queue + per-package-maintainer model doesn't scale to thousands of new packages (the only sane shape would be *bundling*, à la `fonts-noto`). **Sweet spot:** publish clean, declarative build-from-source recipes (the manifest + runner) that Debian's font maintainers can consume — recipes, not packages.

Overlap counts and effort/NEW-queue figures here are estimates from how Debian's process works; a deeper factual pass (exact GF-in-Debian count, Fonts Task Force policy, bundling precedent) is pending.

Drafted by an AI agent (Claude Opus 4.8) under the guidance of @felipesanches.